Artificial Intelligence

Dr. Suman Kumar Maji

Indian Institute of Technology Patna

Search Algorithms

Introduction

- Search algorithms in artificial intelligence are significant.
- They provide solutions to many artificial intelligence-related issues.
- A search issue comprises the search space, start state, and goal state.
- By evaluating scenarios and alternatives, search algorithms assist AI agents in achieving the goal state.

Search Algorithm Terminologies

- **Search:** Searching solves a search issue in a given space step by step. Three major factors can influence a search issue.
- **Search Space:** A search space is a collection of potential solutions a system may have.
- **Start State:** The jurisdiction where the agent starts the search.
- **Goal Test:** A function that examines the current state and returns whether or not the goal state has been attained.
- **Search Tree:** A search tree is a tree representation of a search issue. The node at the root of the search tree corresponds to the initial condition.

Properties of Search Algorithms

- The four important properties of search algorithms in artificial intelligence for comparing their efficiency are as follows:
- **Completeness** – A search algorithm is said to be complete if it guarantees to yield a solution for any random input if at least one solution exists.
- **Optimality** – A solution discovered for an algorithm is considered optimal if it is assumed to be the best solution (lowest path cost) among all other solutions.
- **Time Complexity** – It measures how long an algorithm takes to complete its job.
- **Space Complexity** – The maximum storage space required during the search, as determined by the problem's complexity.
- These characteristics often contrast the effectiveness of various search algorithms in artificial intelligence.

Importance of Search Algorithms in Artificial Intelligence

- **Solving problems:** Applications for route planning, like Google Maps, are one real-world illustration of how search algorithms in AI are utilized to solve problems. These programs employ search algorithms to determine the quickest or shortest path between two locations.
- **Search programming:** Many AI activities can be coded in terms of searching, which improves the formulation of a given problem's solution.
- **Goal-based agents:** Goal-based agents' efficiency is improved through search algorithms in artificial intelligence. These agents look for the most optimal course of action that can offer the finest resolution to an issue to solve it.
- **Neural network systems:** The search for connection weights that will result in the required input-output mapping is improved by search algorithms in AI.

Types of Search Strategies

- In Artificial Intelligence, search techniques are broadly classified into **two types**:

Uninformed Search (Blind Search)
Informed Search (Heuristic Search)

Uninformed Search:

- Uses no additional information about the goal.
- Has knowledge only of the problem definition.
- Explores the search space blindly.

Informed Search:

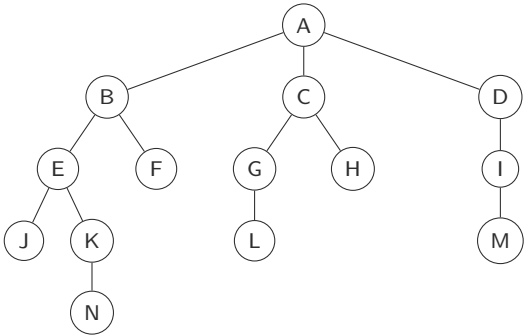
- Uses additional knowledge called a **heuristic**.
- Estimates how close a node is to the goal.
- Expands the most promising node first.

Breadth First Search (BFS)

Breadth First Search (BFS)

- Breadth-First Search (BFS) is a graph traversal algorithm.
- It explores the graph in a **level-by-level** manner.
- The algorithm starts from a given **source node**.
- All immediate neighbors of the source are visited first.
- Then, neighbors of those nodes are explored sequentially.
- BFS is widely used to find **shortest paths** in unweighted graphs.
- It is also useful for exploring **tree-like structures**.

Breadth First Search (BFS)-Example



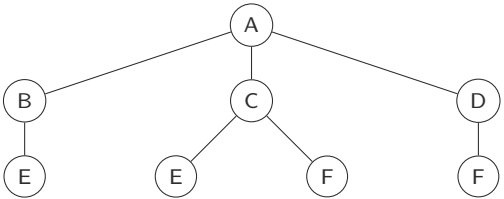
Levels: 0–4

Queue Processing (FIFO)

→ A
~~A~~ → B C D
~~B~~ → C D E F
~~C~~ → D E F G H
~~D~~ → E F G H I
~~E~~ → F G H I J K
~~F~~ → G H I J K
~~G~~ → H I J K L
~~H~~ → I J K L
~~I~~ → J K L M
~~J~~ → K L M
~~K~~ → L M N
~~L~~ → M N
~~M~~ → N
~~N~~ → ∅

BFS Order:
A, B, C, D, E, F, G, H, I, J, K, L, M, N

Breadth First Search (BFS) – Duplicate Nodes Example



Note: E and F appear multiple times

Queue Processing (FIFO)

$\rightarrow$ A
~~A~~ $\rightarrow$ B C D
~~B~~ $\rightarrow$ C D E
~~C~~ $\rightarrow$ D E F (E already in queue)
~~D~~ $\rightarrow$ E F (F already in queue)
~~E~~ $\rightarrow$ F
~~F~~ $\rightarrow$ $\emptyset$

BFS Order:
A, B, C, D, E, F

Key Point:
Duplicate nodes are ignored once visited.

Properties of Breadth First Search (BFS)

- **Shallowest-first search:** BFS explores the graph **level by level**, therefore it always finds the **shallowest (minimum depth)** node first.
- **Complete:** BFS is a **complete** search algorithm. It will always find a solution if one exists, irrespective of the element being searched.
- Since BFS searches level by level, it checks for the **goal state (desired node)** at each level.
- Once the goal node is found, the solution path is obtained by **backtracking using parent pointers**. *Example: If G is the goal node, its parent is C and C's parent is A, then the path is A–C–G.*

Properties of Breadth First Search (BFS)

- **Optimal:** BFS returns the **shortest path** when all edges have **equal cost (unit weight)**. With unequal costs, BFS may not be optimal.
- **Time Complexity:**

  Graph traversal: $O(V + E)$

  State-space search: $O\left(b^d\right)$

  Example (from BFS diagram): In the given BFS example, each node has at most 3 children ($b = 3$) and node **G** is at depth 2. Hence, nodes explored $= 3^2 = 9$.

- **Space Complexity:** $O\left(b^d\right)$ due to queue storage. For the given BFS example, space required up to node **G** is approximately $3^2 = 9$ nodes.
- BFS uses a **FIFO queue** for exploration.

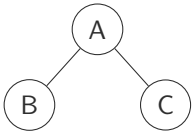
Depth First Search (DFS)

Depth First Search (DFS)

- Depth-First Search (DFS) is a graph traversal algorithm.
- It explores the graph by going **as deep as possible** first.
- The algorithm starts from a given **source node**.
- It visits a node and then explores one of its unvisited neighbors.
- This process continues until no unvisited neighbor remains.
- DFS is commonly used for **path exploration** and backtracking problems.
- It is also useful for exploring **tree and graph structures**.

Stack Convention with Example

Tree Structure



Convention:
Clockwise choice among children.
Pushing direction **left to right**.
(Stack push order: C, then B)

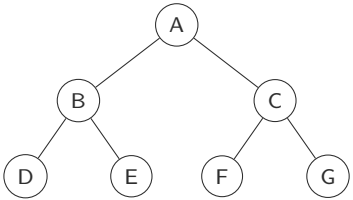
Stack Operations

Push A
Stack:$\rightarrow [A]$

Pop A, Push C, B
Stack:$\rightarrow [B, C]$

Stack format: $\rightarrow [Top, \ldots, Bottom]$
Blue = Top of stack
Red = Bottom of stack

Depth First Search (DFS) – Example 1



Start Node: A

Convention:
Clockwise choice among children.
Pushing direction **left to right**.
Stack format: **[Top,** . . . , *Bottom*]

Stack Processing (LIFO)

Push A
Stack: [A]

Pop A, Push C, B
Stack: [B, C]

Pop B, Push E, D
Stack: [D, E, C]

Pop D (no child)
Stack: [E, C]

Pop E (no child)
Stack: [C]

Pop C, Push G, F
Stack: [F, G]

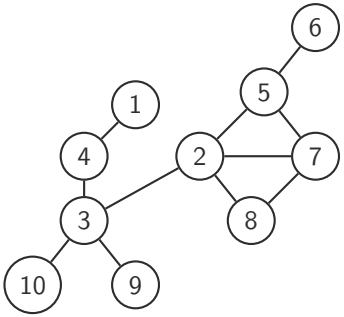
Pop F (no child)
Stack: [G]

Pop G (no child)
Stack: ∅

DFS Traversal Order:

A, B, D, E, C, F, G

Depth First Search (DFS) – Example 2



Start Node = 1

Convention:

Clockwise selection of adjacent nodes.
Pushing direction **left to right**.
Stack shown as **[Top,** . . . , *Bottom*].

DFS Traversal Order:

$1 \to 4 \to 3 \to 10 \to 9 \to 2 \to 8 \to 7 \to 5 \to 6$

DFS Stack Processing (LIFO)

Push 1
Stack: [1]

Pop 1, Push 4
Stack: [4]

Pop 4, Push 3
Stack: [3]

Pop 3, Push 2, 9, 10
Stack: [10, 9, 2]

Pop 10
Stack: [9, 2]

Pop 9
Stack: [2]

Pop 2, Push 5, 7, 8
Stack: [8, 7, 5]

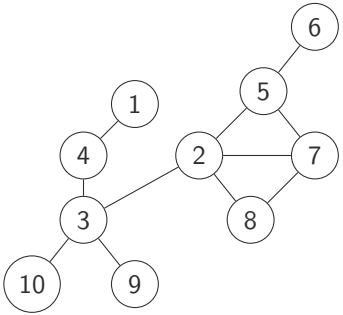
Pop 8
Stack: [7, 5]

Pop 7
Stack: [5]

Pop 5, Push 6
Stack: [6]

Pop 6
Stack: $\emptyset$

DFS: DFS Spanning Tree – Example 2 (Spanning Tree)



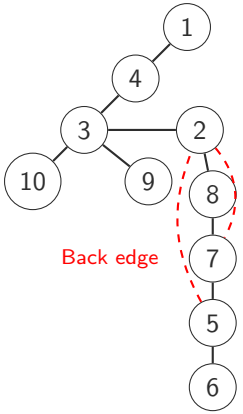
Original Graph

DFS Traversal Order:

$1 \rightarrow 4 \rightarrow 3 \rightarrow 10 \rightarrow 9 \rightarrow 2 \rightarrow 8 \rightarrow 7 \rightarrow 5 \rightarrow 6$

Start Node = 1

DFS Spanning Tree



Properties of Depth First Search (DFS)

- **Uninformed Search Technique:** DFS does not use any heuristic or additional information about the goal.
- **Uses Stack (LIFO):** DFS uses a **stack** data structure (explicitly or via recursion).
- **Deepest Node First:** DFS explores the **deepest possible node** along a path before backtracking.
- **Incomplete:** DFS is not complete in infinite-depth or cyclic graphs, as it may get stuck exploring one path indefinitely.
- **Non-Optimal:** DFS does not guarantee the shortest or optimal solution.
- **Time Complexity:**
- For graph traversal: $O(V + E)$
- For state-space search: $O(b^m)$
- **Space Complexity:** $O(bm)$ or $O(m)$, where m is the maximum depth of the tree.

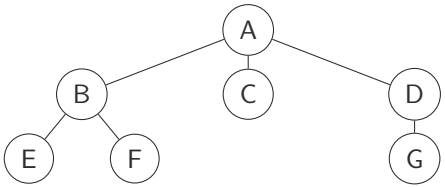
BFS vs DFS

Terminologies:

- b **(Branching Factor):** Maximum number of children that any node in the search tree can generate.
- d **(Solution Depth):** Depth (level) at which the shallowest goal node is found in the search tree.
- m **(Maximum Depth):** Maximum depth of the search space.

Criterion	DFS	BFS	Explanation
Complete?	No	Yes	BFS explores level by level; DFS may get stuck in deep paths.
Time (Tree)	$O(b^m)$	$O\left(b^d\right)$	DFS may explore up to max depth m; BFS explores all nodes up to depth d.
Space (Tree)	$O(bm)$ / $O(m)$	$O\left(b^d\right)$	DFS stores only current path; BFS stores entire frontier.
Time (Graph)	$O(V+E)$	$O(V+E)$	Each vertex and each edge is visited at most once.
Space (Graph)	$O(V)$	$O(V)$	Visited set + stack (DFS) or queue (BFS).
Optimal?	No	Yes	BFS finds shortest path in unweighted graphs.

Depth First Search (DFS) – Complexity (Example)



Example Tree , G = Goal Node

*** If one more level is added:**
Depth becomes $m = 3$

Time Complexity $= O(3^3) = O(27)$

Space Complexity $= O(3 \times 3) = O(9)$

- **Branching Factor:** $b = 3$ (maximum number of children of any node)
- **Maximum Depth:** $m = 2$ (goal node G is at depth 2)
- **Time Complexity:**

$$O(b^m) = O(3^2) = O(9)$$

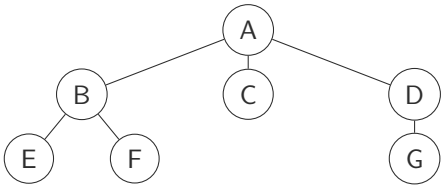
DFS may explore all nodes up to depth m.

- **Space Complexity:**

$$O(b \cdot m) = O(3 \times 2) = O(6)$$

Space is required for the recursion stack and siblings.

Breadth First Search (BFS) – Complexity (Example)



Example Tree , G = Goal Node

- **Branching Factor:** $b = 3$ (maximum children per node)
- **Solution Depth:** $d = 2$
- **Time Complexity:**

$$O\left(b^d\right) = O\left(3^2\right) = O\left(9\right)$$

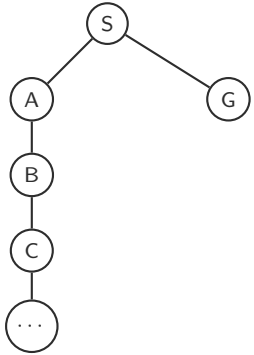
BFS explores all nodes level by level until depth d.

- **Space Complexity:**

$$O\left(b^d\right) = O\left(3^2\right) = O\left(9\right)$$

BFS stores all nodes at the deepest level in the queue.

BFS vs DFS – Infinite Graph Example



Infinite Graph , G = Goal Node

Parameters:

$$b = 2, \quad d = 1, \quad m = \infty$$

BFS:

- Reaches goal G at depth $d = 1$
- Time: $O\left(b^d\right) = O(2)$
- Space: $O\left(b^d\right) = O(2)$
- Complete and optimal

DFS:

- Explores infinite branch
- Time: $O(b^m) = O(\infty)$
- Space: $O(bm) = O(\infty)$
- Not complete

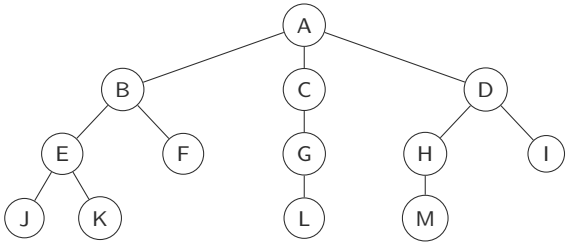
Depth-Limited Search (DLS)

Depth-Limited Search (DLS)

- **Depth-Limited Search (DLS)** is a **Depth-First Search (DFS)** with a fixed depth limit.
- Expands nodes only up to a predefined **maximum depth** L.
- Nodes beyond the depth limit are **not explored**.
- Prevents the problem of **infinite paths** in DFS.
- Acts as a **building block** for Iterative Deepening DFS (IDDFS).
- **Complete** only if the depth limit L is greater than or equal to the goal depth.
- **Not optimal** in general (may not find the shallowest goal).

Key Idea:
DFS with a controlled depth boundary.

Depth-Limited Search (DLS) – Depth Limit = 1 (Goal Not Found)



Goal Node: I

Convention:
Clockwise selection of adjacent nodes.
Pushing direction **left to right**.
Stack shown as **[Top,** . . . , *Bottom*].

DLS Stack (Depth Limit = 1)

Push A
Stack: [A]

Pop A, Push D, C, B
Stack: [B, C, D]

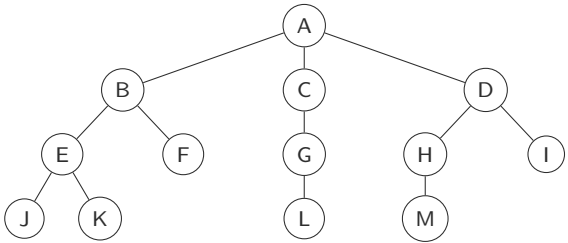
Pop B (depth limit reached)

Pop C (depth limit reached)

Pop D (depth limit reached)

Goal Found? No

Depth-Limited Search (DLS) – Depth Limit = 2 (Goal Found)



Goal Node: I

Convention:
Clockwise selection of adjacent nodes.
Pushing direction **left to right**.
Stack shown as **[Top,** . . . , *Bottom*].

DLS Stack (Depth Limit = 2)

Push A
Stack: [A]

Pop A, Push D, C, B
Stack: [B, C, D]

Pop B, Push F, E
Stack: [E, F, C, D]

Pop E (depth limit reached)

Pop F (depth limit reached)

Pop C, Push G
Stack: [G, D]

Pop G (depth limit reached)

Pop D, Push I, H
Stack: [H, I]

Pop H (depth limit reached)

Pop I (Goal Found)

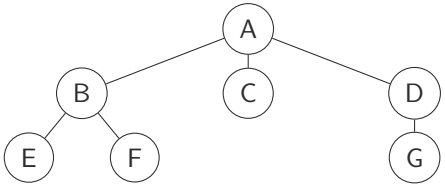
Goal Found? Yes

Traversal Order :
A → B → E → F → C → G → D → H → I

Properties of Depth-Limited Search (DLS)

- **Uninformed Search Technique:** Uses no heuristic or goal distance information.
- **Variant of DFS:** Depth-First Search with a predefined depth limit.
- **Fixed Depth Limit:** Expands nodes only up to a maximum depth L.
- **Incomplete in General:** Fails to find the goal if the solution depth is greater than the depth limit L.
- **Not Optimal:** May miss the shallowest solution even with unit costs.
- **Time Complexity:** $O\left(b^L\right)$, where b is branching factor and L is depth limit.
- **Space Complexity:** $O\left(bL\right)$, same as DFS due to stack-based exploration.

Depth-Limited Search (DLS) – Complexity (Example)



Example Tree
G = Goal Node

- **Branching Factor:** $b = 3$
- **Depth Limit:** $L = 2$
- **Time Complexity (Goal within limit):** $O\left(b^L\right) = O\left(3^2\right) = O\left(9\right)$; explores nodes only up to depth L.
- **If Goal Node is beyond limit L:** DLS fails to find the goal but still explores all nodes up to depth L; time complexity remains $O\left(b^L\right) = O\left(3^2\right) = O\left(9\right)$.
- **Space Complexity:** $O\left(b \cdot L\right) = O\left(3 \times 2\right) = O\left(6\right)$ using DFS stack up to depth limit.

Iterative Deepening DFS (IDDFS)

Iterative Deepening Depth-First Search (IDDFS)

- Iterative Deepening Depth-First Search (IDDFS) is a graph traversal algorithm that combines the advantages of Depth-First Search (DFS) and Breadth-First Search (BFS).
- It is optimal and complete like BFS, has time complexity of BFS i.e., $O\left(b^d\right)$ and space complexity of DFS i.e. $O(bd)$ (which is less than BFS).
- IDDFS performs a depth-limited DFS and gradually increases the depth limit until a solution is found.
- The search is repeated until the goal state is found or all nodes have been explored.

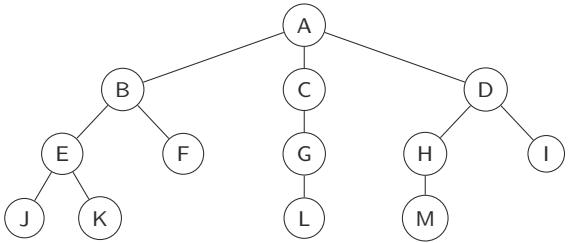
Steps of Iterative Deepening DFS (IDDFS)

- Set the initial depth limit to 0.
- Perform Depth-First Search (DFS) up to the current depth limit.
- If the goal state is found, return the solution.
- If the goal state is not found and the maximum depth has not been reached, increase the depth limit and repeat the steps 2-4.
- If the goal state is not found and the maximum depth is reached, terminate the search and return failure.

Steps of Iterative Deepening DFS (IDDFS)

- **Step 1:** Set the initial depth limit to 0.
- **Step 2:** Perform Depth-First Search (DFS) up to the current depth limit.
- **Step 3:** If the goal state is found, return the solution.
- **Step 4:** If the goal state is not found and the maximum depth has not been reached, increase the depth limit and repeat steps 2-4.
- **Step 5:** If the goal state is not found and the maximum depth is reached, terminate the search and return failure.

IDDFS – Depth Limit = 0



Goal Node: M

Convention:
Clockwise selection of adjacent nodes.
Pushing direction **left to right**.
Stack shown as **[Top,** . . . , *Bottom*].

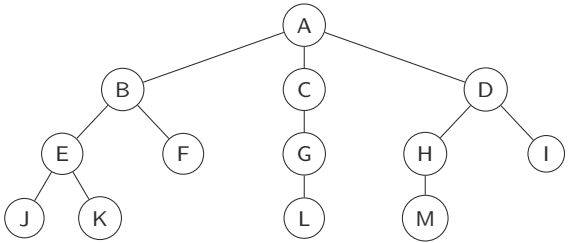
DFS Stack (Depth Limit = 0)

Push A
Stack: [A]

Pop A (depth limit reached)
Stack: ∅

Goal Found? No

IDDFS – Depth Limit = 1



Goal Node: M

Convention:
Clockwise selection of adjacent nodes.
Pushing direction **left to right**.
Stack shown as **[Top,** . . . , *Bottom*].

DFS Stack (Depth Limit = 1)

Push A
Stack: [A]

Pop A, Push D, C, B
Stack: [B, C, D]

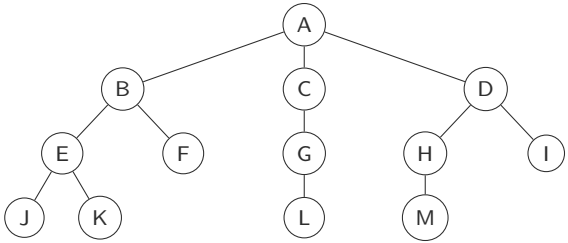
Pop B (depth limit reached)
Stack: [C, D]

Pop C (depth limit reached)
Stack: [D]

Pop D (depth limit reached)
Stack: ∅

Goal Found? No

IDDFS – Depth Limit = 2



Goal Node: M

Convention:
Clockwise selection of adjacent nodes.
Pushing direction **left to right**.
Stack shown as **[Top,** . . . , *Bottom*].

DFS Stack (Depth Limit = 2)

Push A
Stack: [A]

Pop A, Push D, C, B
Stack: [B, C, D]

Pop B, Push F, E
Stack: [E, F, C, D]

Pop E (depth limit reached)
Stack: [F, C, D]

Pop F (depth limit reached)
Stack: [C, D]

Pop C, Push G
Stack: [G, D]

Pop G (depth limit reached)
Stack: [D]

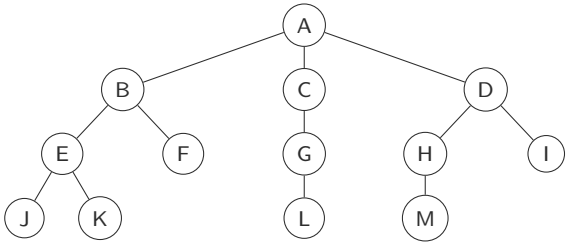
Pop D, Push I, H
Stack: [H, I]

Pop H (depth limit reached)
Stack: [I]

Pop I (depth limit reached)
Stack: ∅

Goal Found? No

IDDFS – Depth Limit = 3 (Part1)



Goal Node: M

Convention:
Clockwise selection of adjacent nodes.
Pushing direction **left to right**.
Stack shown as **[Top,** . . . , *Bottom*].

DFS Stack (Depth Limit = 3)

Push A
Stack: [A]

Pop A, Push D, C, B
Stack: [B, C, D]

Pop B, Push F, E
Stack: [E, F, C, D]

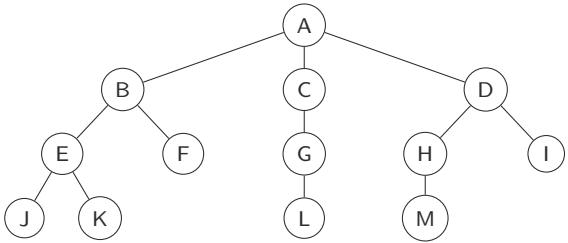
Pop E, Push K, J
Stack: [J, K, F, C, D]

Pop J (depth limit reached)
Stack: [K, F, C, D]

Pop K (depth limit reached)
Stack: [F, C, D]

Pop F (depth limit reached)
Stack: [C, D]

IDDFS – Depth Limit = 3 (Part2)



Goal Node: M

Convention:
Clockwise selection of adjacent nodes.
Pushing direction **left to right**.
Stack shown as **[Top,** . . . , *Bottom*].

DFS Stack (Depth Limit = 3)

Pop C, Push G
Stack: [G, D]

Pop G, Push L
Stack: [L, D]

Pop L (depth limit reached)
Stack: [D]

Pop D, Push I, H
Stack: [H, I]

Pop H, Push M
Stack: [M, I]

Pop M (Goal Found)

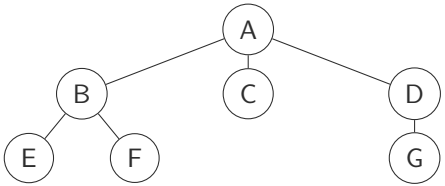
Goal Found? Yes

Traversal Order:
A, B, E, J, K, F, C, G, L, D, H, M

Properties of Iterative Deepening DFS (IDDFS)

- **Uninformed Search Technique:** Uses no heuristic or goal information.
- **Combination of DFS and BFS:** Uses DFS-style stack with BFS-style increasing depth limits.
- **Repeated Depth-Limited Search:** Performs DFS multiple times with depth limits $0, 1, 2, \ldots$ until the goal is found.
- **Complete:** Guaranteed to find a solution if branching factor is finite.
- **Optimal (Unit Cost):** Finds the shallowest solution when all edge costs are equal.
- **Time Complexity:** $O\left(b^d\right)$, where b is branching factor and d is solution depth.
- **Space Complexity:** $O(bd)$, same as DFS and much lower than BFS.

Iterative Deepening DFS (IDDFS) – Complexity (Example)



Example Tree , G = Goal Node

- **Branching Factor:** $b = 3$ (maximum children per node)
- **Solution Depth:** $d = 2$
- **Time Complexity:**

$$O\left(b^d\right) = O\left(3^2\right) = O\left(9\right)$$

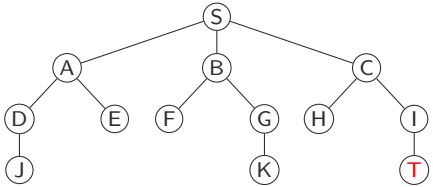
IDDFS repeats DFS for depth limits 0 to d, but total cost remains $O\left(b^d\right)$.

- **Space Complexity:**

$$O\left(b \cdot d\right) = O\left(3 \times 2\right) = O\left(6\right)$$

Uses DFS stack space at each iteration.

Search Algorithms – Traversal Question



Start Node: S
Goal Node: T

Conventions:
Clockwise selection of adjacent nodes.
Pushing direction **left to right**.
DFS, DLS and IDDFS use **Stack (LIFO)**.
BFS uses **Queue (FIFO)**.

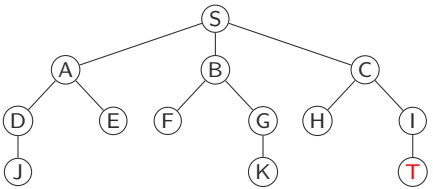
Question:
Perform **Breadth First Search (BFS)** on the given graph and write the traversal order.

Perform **Depth First Search (DFS)** on the given graph and write the traversal order.

Perform **Depth-Limited Search (DLS)** with **Depth Limit = 2**.

Perform **Iterative Deepening Depth First Search (IDDFS)** until the goal node is found.

Search Algorithms – Solution



Start Node: S
Goal Node: T

Final Traversal Results

BFS:
S → A → B → C → D → E → F → G → H → I → J → K → T

DFS:
S → A → D → J → E → B → F → G → K → C → H → I → T

DLS (Depth Limit = 2):
Goal not found.

IDDFS:
Goal found at depth = 3
Traversal order same as DFS until goal is reached.

Uniform Cost Search(UCS)

Algorithm: Uniform Cost Search

- **Uniform Cost Search assumes that all operators have a cost.** Each node n has cost $C[n]$ (cost from start). UCS selects the minimum cost state.

1. **[Initialize]** OPEN={s}, CLOSED={}, $C[s] = 0$
2. **[Fail]** If OPEN={}, terminate with Failure.
3. **[Select]** Select minimum cost state n from OPEN; move n to CLOSED.
4. **[Terminate]** If $n \in G$, terminate with Success.
5. **[Expand]**

For each successor m of n:
If $m \notin (OPEN \cup CLOSED)$

$$C[m] = C[n] + C(n, m)$$

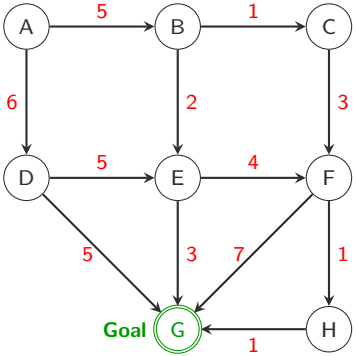
Insert m into OPEN
If $m \in (OPEN \cup CLOSED)$

$$C[m] = \min\{C[m], C[n] + C(n, m)\}$$

If cost decreased and $m \in CLOSED$, move to OPEN

6. **[Loop]** Go to Step 2.

Uniform Cost Search (UCS)

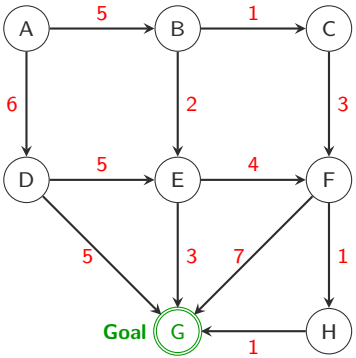


Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome

Uniform Cost Search (UCS)

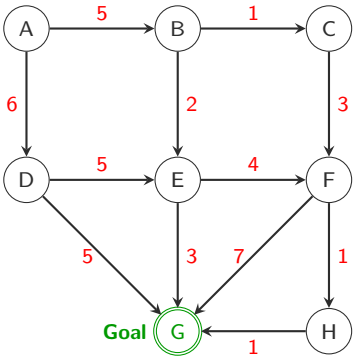


Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome
1	C={},O={(A,0)}

Uniform Cost Search (UCS)

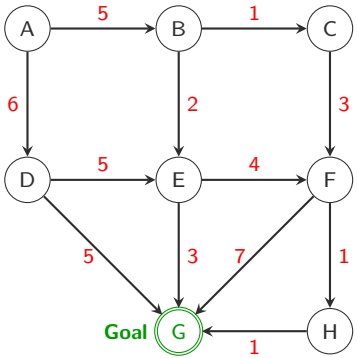


Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome
1	C={},O={(A,0)}
2.	C={A},O={(B,5), (D,6)}

Uniform Cost Search (UCS)

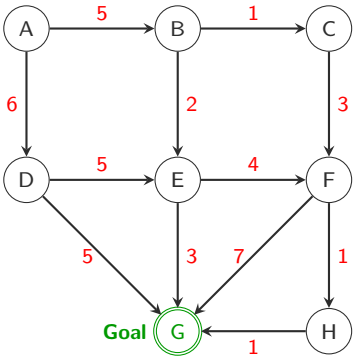


Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome
1	C={} ,O={(A,0)}
2.	C={A},O={(B,5), (D,6)}
3.	C={A,B}, O={(D,6), (C,6), (E,7)}

Uniform Cost Search (UCS)

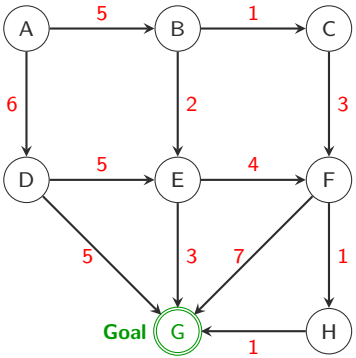


Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome
1	C={} ,O={(A,0)}
2.	C={A},O={(B,5), (D,6)}
3.	C={A,B}, O={(D,6), (C,6), (E,7)}
4.	C={A,B,D}, O={(C,6), (E,7), (G,11)}

Uniform Cost Search (UCS)

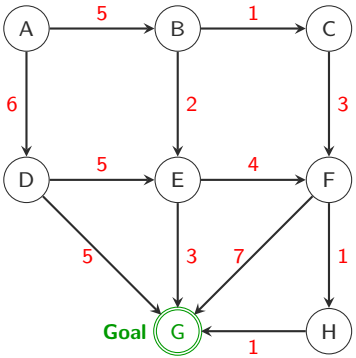


Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome
1	C={} ,O={(A,0)}
2.	C={A},O={(B,5), (D,6)}
3.	C={A,B}, O={(D,6), (C,6), (E,7)}
4.	C={A,B,D}, O={(C,6), (E,7), (G,11)}
5.	C={A,B,D,C}, O={(E,7), (G,11), (F,9)}

Uniform Cost Search (UCS)

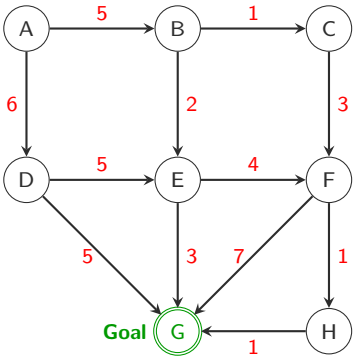


Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome
1	C={} ,O={(A,0)}
2.	C={A},O={(B,5), (D,6)}
3.	C={A,B}, O={(D,6), (C,6), (E,7)}
4.	C={A,B,D}, O={(C,6), (E,7), (G,11)}
5.	C={A,B,D,C}, O={(E,7), (G,11) , (F,9)}
6.	C={A,B,D,C,E} , O={ (G,10),(F,9)}

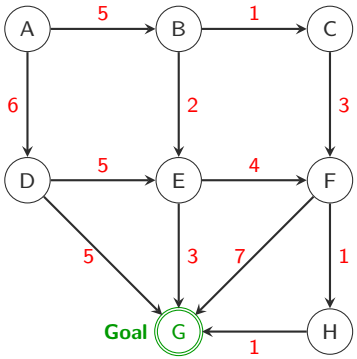
Uniform Cost Search (UCS)



(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome
1	C={} ,O={(A,0)}
2.	C={A},O={(B,5), (D,6)}
3.	C={A,B}, O={(D,6), (C,6), (E,7)}
4.	C={A,B,D}, O={(C,6), (E,7), (G,11)}
5.	C={A,B,D,C}, O={(E,7), (G,11) , (F,9)}
6.	C={A,B,D,C,E} , O={ (G,10),(F,9)}
7.	C={A,B,D,C,E,F}, O={(G,10), (H,10)}

Uniform Cost Search (UCS)

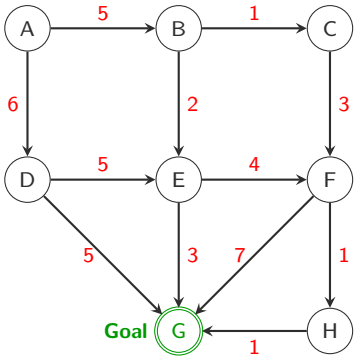


Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome
1	C={} ,O={(A,0)}
2.	C={A},O={(B,5), (D,6)}
3.	C={A,B}, O={(D,6), (C,6), (E,7)}
4.	C={A,B,D}, O={(C,6), (E,7), (G,11)}
5.	C={A,B,D,C}, O={(E,7), (F,9), (G,11)}
6.	C={A,B,D,C,E} , O={(F,9), (G,10)}
7.	C={A,B,D,C,E,F}, O={(G,10), (H,10)}
8.	C={A,B,D,C,E,F,G}, O={ (H,10)}

Uniform Cost Search (UCS)



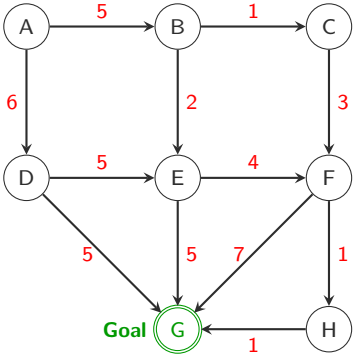
Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

St.	Outcome
1	C={} ,O={(A,0)}
2.	C={A},O={(B,5), (D,6)}
3.	C={A,B}, O={(D,6), (C,6), (E,7)}
4.	C={A,B,D}, O={(C,6), (E,7), (G,11)}
5.	C={A,B,D,C}, O={(E,7), (F,9), (G,11)}
6.	C={A,B,D,C,E} , O={(F,9), (G,10)}
7.	C={A,B,D,C,E,F}, O={(G,10), (H,10)}
8.	C={A,B,D,C,E,F,G}, O={ (H,10)}
9	Goal G found

Terminate with cost = **10**

Search Algorithms – Practice Question



Start Node: A

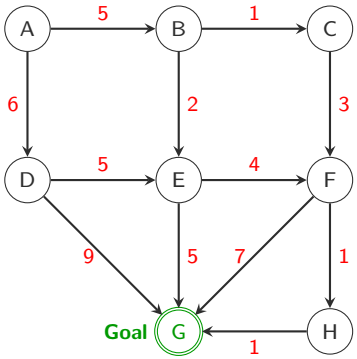
Question:
Perform **Uniform Cost Search (UCS)** from Start Node A to Goal Node G.

- Show step-by-step execution.
- Show the OPEN list at each step.
- Show the CLOSED list at each step.

(node, g(n)) O – OPEN C – CLOSED

Step	CLOSED	OPEN

Uniform Cost Search (UCS)- Practice Question (solution)



Start Node: A

(node, g(n)), O–OPEN, C–CLOSED

1. C={} ,O={(A,0)}
2. C={A} , O={(B,5),(D,6)}
3. C={A,B} ,O={(D,6),(C,6),(E,7)}
4. C={A,B,D} ,O={(C,6),(E,7),(G,15)}
5. C={A,B,D,C} , O={(E,7),(G,15),(F,9)}
6. C={A,B,D,C,E} ,O={(G,12),(F,9)},
7. C={A,B,D,C,E,F},O={(G,12),(H,10)},
8. C={A,B,D,C,E,F,H} ,O={(G,11)}
9. C={A,B,D,C,E,F,H,G} ,O={}
10. **Goal G found**

Terminate with cost = **11**

Thank you for your attention!